

Data Gathering, Cleaning, and Preprocessing Report

Domain: Banking, Financial Services and Insurance (BFSI) — Personal Loan Applicant Data

Prepared by: Kunal | Week 1 Internship Task | Date: September 2026

1. Objective

The goal of this task was to source a dataset relevant to a real-world business domain, explore its structure, identify data quality issues, and apply systematic cleaning and preprocessing techniques so that the dataset is analysis-ready. Given a career interest in the BFSI (Banking, Financial Services and Insurance) sector, a **personal loan applicant dataset** was used — the kind of data a retail bank's credit/underwriting team would work with to assess loan eligibility.

2. Dataset Description

The dataset represents individual loan applications submitted to a bank, and mirrors the structure of publicly available loan-eligibility datasets commonly used for BFSI analytics practice (e.g. the widely used "Loan Prediction" dataset format). It was constructed to reflect the same column structure, data types, and realistic data-quality problems (missing values, inconsistent labels, invalid entries, outliers, duplicates) that are typically encountered in raw operational banking data, so that the full cleaning workflow could be demonstrated end-to-end.

2.1 Original (Raw) Dataset Structure

Raw shape: **2025 rows × 15 columns**

- **Customer_ID** — unique applicant identifier
- **Gender, Marital_Status, Education, Self_Employed** — categorical demographic fields
- **Age** — applicant age in years
- **ApplicantIncome, CoapplicantIncome** — monthly income (numeric)
- **LoanAmount** — requested loan amount (in thousands)
- **Loan_Amount_Term** — loan repayment term (days)
- **Credit_History** — 1 = good credit history, 0 = poor
- **Property_Area** — Urban / Semiurban / Rural
- **Credit_Score** — bureau credit score
- **Application_Date** — date of loan application

- **Loan_Status** — Approved / Rejected

3. Initial Data Exploration

An initial inspection of the raw file was performed to understand shape, data types, missing values, and duplication before any cleaning was applied.

3.1 Raw Data Types

Column	Value
Customer_ID	str
Gender	str
Age	float64
Marital_Status	str
Education	str
Self_Employed	str
ApplicantIncome	float64
CoapplicantIncome	float64
LoanAmount	float64
Loan_Amount_Term	int64
Credit_History	float64
Property_Area	str
Credit_Score	float64
Application_Date	str
Loan_Status	str

3.2 Missing Values (Raw)

Column	Missing Count
Customer_ID	0
Gender	85
Age	60
Marital_Status	128
Education	0
Self_Employed	208
ApplicantIncome	45
CoapplicantIncome	0
LoanAmount	75
Loan_Amount_Term	0
Credit_History	223

Property_Area	0
Credit_Score	42
Application_Date	0
Loan_Status	0

Exact duplicate rows found: **24**. Rows sharing a duplicate **Customer_ID** (same customer appearing more than once, sometimes with conflicting data): **25**.

4. Data Anomalies Identified

- **Missing values** across Gender, Age, Marital_Status, Self_Employed, ApplicantIncome, LoanAmount, Credit_History, and Credit_Score.
- **Inconsistent category labels** — e.g. Gender recorded as 'Male', 'M', 'male'; Property_Area as 'Urban', 'URBAN', ' Semiurban '; Loan_Status as 'Y'/'Approved'.
- **Invalid / impossible values** — negative or unrealistic ages (e.g. -5, 200 years), negative loan amounts, credit scores outside the valid 300–900 bureau range.
- **Inconsistent date formats** in Application_Date (ISO, DD/MM/YYYY, MM-DD-YYYY, and DD-Mon-YYYY all present in the same column).
- **Outliers** in ApplicantIncome and LoanAmount, including a small number of extreme high-income entries roughly 15x the typical value.
- **Duplicate records** — exact duplicate rows and repeated Customer_ID values.
- **Incorrect data types** — numeric fields (Age, Credit_History, Credit_Score) stored as floats due to embedded missing values; dates stored as free-text strings.

5. Cleaning & Preprocessing Methodology

5.1 Duplicate Removal

Exact duplicate rows were dropped (24 removed). Rows with a repeated Customer_ID were then de-duplicated, keeping the first occurrence, since a customer should map to one application record in this cleaned view (1 additional row removed).

5.2 Standardizing Categorical Values

All text categories were trimmed of whitespace and normalized to a consistent case/label (e.g. 'M' / 'male' → 'Male'; 'URBAN' / ' Semiurban ' → 'Urban' / 'Semiurban'; 'Y' → 'Approved' for Loan_Status). This ensures each category is counted once during analysis instead of being split across near-duplicate labels.

5.3 Parsing Inconsistent Dates

Application_Date contained four different date formats. A format-detection routine was applied to parse every value into a single standard ISO format (YYYY-MM-DD). Unparseable dates after this step: **0**.

5.4 Correcting Invalid / Out-of-Range Values

Domain rules were applied to catch impossible entries, which were then treated as missing (to be handled in the imputation step) rather than silently kept:

- Age outside the plausible 18–100 year range → 66 values flagged and set to missing.
- Negative LoanAmount values → 10 values flagged and set to missing.
- Credit_Score outside the standard 300–900 bureau range → 28 values flagged and set to missing.

5.5 Outlier Treatment (IQR Method)

The Interquartile Range (IQR) method (values beyond $Q1 - 1.5 \times IQR$ or $Q3 + 1.5 \times IQR$) was used to detect outliers in ApplicantIncome and LoanAmount. Rather than deleting these rows — which would discard genuine high-income/high-loan applicants and shrink the dataset — values were **capped (winsorized)** at the IQR bounds, preserving sample size while limiting the influence of extreme values on downstream statistics/models.

Column	Lower Bound	Upper Bound	Outliers Capped
ApplicantIncome	-30019.5	137131.87	118
LoanAmount	-6825.0	25775.0	125

5.6 Handling Missing Values

After flagging invalid values as missing, imputation was applied based on each field's distribution and type:

- **Numeric fields** (Age, ApplicantIncome, LoanAmount, Credit_Score) — imputed with the **median**, which is robust to skew and residual outliers.
- **Credit_History** (binary indicator) — imputed with the **mode** (most frequent class).
- **Categorical fields** (Gender, Marital_Status, Self_Employed) — imputed with the **mode** of each column.

Missing value counts before and after imputation:

Column	Missing (before)
Customer_ID	0
Gender	83
Age	126
Marital_Status	127
Education	0
Self_Employed	205
ApplicantIncome	45
CoapplicantIncome	0
LoanAmount	85
Loan_Amount_Term	0
Credit_History	223

Property_Area	0
Credit_Score	69
Application_Date	0
Loan_Status	0

All columns had zero missing values after imputation (verified in Section 6).

5.7 Data Type Conversions

- Age, Credit_History, Loan_Amount_Term, Credit_Score → converted to integer type (no missing values remained, and these are inherently whole-number fields).
- LoanAmount, ApplicantIncome → rounded to 2 decimal places as currency-style floats.
- Application_Date → converted from mixed-format free text to a proper datetime, then standardized to ISO string format (YYYY-MM-DD) for CSV portability.

5.8 Feature Engineering & Normalization

- **Total_Income** = ApplicantIncome + CoapplicantIncome — a single household-income view commonly used in credit underwriting.
- **Loan_to_Income_Ratio** = LoanAmount / Total_Income — a key affordability indicator for loan risk assessment.
- **Min-max normalized copies** (0–1 scale) were added for ApplicantIncome, LoanAmount, Credit_Score, and Total_Income (suffixes *_Normalized*), so the dataset is ready for distance-based or gradient-based models without discarding the original interpretable values.

6. Final Cleaned Dataset

Final shape: **2000 rows × 21 columns** (vs. 2025 rows x 15 columns raw).

6.1 Final Data Types

Column	Value
Customer_ID	str
Gender	str
Age	int64
Marital_Status	str
Education	str
Self_Employed	str
ApplicantIncome	float64
CoapplicantIncome	float64
LoanAmount	float64
Loan_Amount_Term	int64
Credit_History	int64
Property_Area	str
Credit_Score	int64
Application_Date	str
Loan_Status	str
Total_Income	float64
Loan_to_Income_Ratio	float64
ApplicantIncome_Normalized	float64
LoanAmount_Normalized	float64
Credit_Score_Normalized	float64
Total_Income_Normalized	float64

6.2 Final Missing Value Check

All columns confirmed at zero missing values post-cleaning:

Column	Missing Count
Customer_ID	0
Gender	0
Age	0
Marital_Status	0
Education	0
Self_Employed	0

ApplicantIncome	0
CoapplicantIncome	0
LoanAmount	0
Loan_Amount_Term	0
Credit_History	0
Property_Area	0
Credit_Score	0
Application_Date	0
Loan_Status	0
Total_Income	0
Loan_to_Income_Ratio	0
ApplicantIncome_Normalized	0
LoanAmount_Normalized	0
Credit_Score_Normalized	0
Total_Income_Normalized	0

7. Summary of Corrective Measures

Issue	Action Taken	Rationale
Missing values	Median (numeric) / mode (categorical) imputation	Preserves sample size; robust to skew
Invalid values (age, credit score, negative loan amount)	Flagged as missing, then imputed	Avoids fabricating implausible data as 'real'
Inconsistent category labels	Standardized casing/spelling	Prevents category fragmentation in analysis
Inconsistent date formats	Parsed to single ISO format	Enables correct chronological analysis
Outliers (income, loan amount)	Capped via IQR winsorization	Limits distortion without discarding records
Duplicate rows / duplicate customer IDs	Dropped, keeping first occurrence	Avoids double-counting the same application
Incorrect data types	Converted to int/float/datetime as appropriate	Ensures correct downstream computation

8. Conclusion

The raw loan applicant dataset was transformed from a state with missing values, invalid entries, inconsistent formatting, and outliers into a clean, consistently typed, analysis-ready dataset with zero missing values and two engineered features (Total_Income, Loan_to_Income_Ratio) plus normalized numeric columns for modeling. This mirrors the kind of data preparation pipeline expected before exploratory analysis or building a credit-risk / loan-approval prediction model in a BFSI setting — directly relevant to the Data Analyst / Data Scientist roles this internship is preparing for.